

2.6 locale.h

The locale header is useful for setting location specific information.

Variables:

```
struct lconv
```

Macros:

```
NULL
LC_ALL
LC_COLLATE
LC_CTYPE
LC_MONETARY
LC_NUMERIC
LC_TIME
```

Functions:

```
localeconv();
setlocale();
```

2.6.1 Variables and Definitions

The `lconv` structure contains the following variables in any order. The use of this structure is described in **2.6.3 localeconv**.

```
char *decimal_point;
char *thousands_sep;
char *grouping;
char *int_curr_symbol;
char *currency_symbol;
char *mon_decimal_point;
char *mon_thousands_sep;
char *mon_grouping;
char *positive_sign;
char *negative_sign;
char int_frac_digits;
char frac_digits;
char p_cs_precedes;
char p_sep_by_space;
char n_cs_precedes;
char n_sep_by_space;
char p_sign_posn;
char n_sign_posn;
```

The `LC_` macros are described in **2.6.2 setlocale**. `NULL` is the value of a null pointer constant.

2.6.2 setlocale

Declaration:

```
char *setlocale(int category, const char *locale);
```

Sets or reads location dependent information.

category can be one of the following:

LC_ALL Set everything.

LC_COLLATE Affects strcoll and strxfrm functions.

LC_CTYPE Affects all character functions.

LC_MONETARY Affects the monetary information provided by localeconv function.

LC_NUMERIC Affects decimal-point formatting and the information provided by localeconv function.

LC_TIME Affects the strftime function.

A value of "C" for locale sets the locale to the normal C translation environment settings (default). A null value ("") sets the native environment settings. A null pointer (NULL) causes setlocale to return a pointer to the string associated with this category for the current settings (no changes occur). All other values are implementation-specific.

After a successful set, **setlocale** returns a pointer to a string which represents the previous location setting. On failure it returns NULL.

Example:

```
#include<locale.h>
#include<stdio.h>

int main(void)
{
    char *old_locale;

    old_locale=setlocale(LC_ALL,"C");
    printf("The preivous setting was %s.\n",old_locale);
    return 0;
}
```

2.6.3 localeconv

Declaration:

```
struct lconv *localeconv(void);
```

Sets the structure **lconv** to represent the current location settings.

The string pointers in the structure may point to a null string ("") which indicates that the value is not available. The char types are nonnegative numbers. If the value is **CHAR_MAX**, then the value is not available.

lconv variables:

char *decimal_point Decimal point character used for non-monetary values.

char *thousands_sep Thousands place separator character used for non-monetary values.

<code>char *grouping</code>	A string that indicates the size of each group of digits in non-monetary quantities. Each character represents an integer value which designates the number of digits in the current group. A value of 0 means that the previous value is to be used for the rest of the groups.
<code>char *int_curr_symbol</code>	A string of the international currency symbols used. The first three characters are those specified by ISO 4217:1987 and the fourth is the character which separates the currency symbol from the monetary quantity.
<code>char *currency_symbol</code>	The local symbol used for currency.
<code>char *mon_decimal_point</code>	The decimal point character used for monetary values.
<code>char *mon_thousands_sep</code>	The thousands place grouping character used for monetary values.
<code>char *mon_grouping</code>	A string whose elements define the size of the grouping of digits in monetary values. Each character represents an integer value which designates the number of digits in the current group. A value of 0 means that the previous value is to be used for the rest of the groups.
<code>char *positive_sign</code>	The character used for positive monetary values.
<code>char *negative_sign</code>	The character used for negative monetary values.
<code>char int_frac_digits</code>	Number of digits to show after the decimal point in international monetary values.
<code>char frac_digits</code>	Number of digits to show after the decimal point in monetary values.
<code>char p_cs_precedes</code>	If equal to 1, then the <code>currency_symbol</code> appears before a positive monetary value. If equal to 0, then the <code>currency_symbol</code> appears after a positive monetary value.
<code>char p_sep_by_space</code>	If equal to 1, then the <code>currency_symbol</code> is separated by a space from a positive monetary value. If equal to 0, then there is no space between the <code>currency_symbol</code> and a positive monetary value.
<code>char n_cs_precedes</code>	If equal to 1, then the <code>currency_symbol</code> precedes a negative monetary value. If equal to 0, then the <code>currency_symbol</code> succeeds a negative monetary value.
<code>char n_sep_by_space</code>	If equal to 1, then the <code>currency_symbol</code> is separated by a space from a negative monetary value. If equal to 0, then there is no space between the <code>currency_symbol</code> and a negative monetary value.
<code>char p_sign_posn</code>	Represents the position of the <code>positive_sign</code> in a positive monetary value.
<code>char n_sign_posn</code>	Represents the position of the <code>negative_sign</code> in a negative monetary value.

The following values are used for `p_sign_posn` and `n_sign_posn`:

- 0 Parentheses encapsulate the value and the `currency_symbol`.
- 1 The sign precedes the value and `currency_symbol`.
- 2 The sign succeeds the value and `currency_symbol`.
- 3 The sign immediately precedes the value and `currency_symbol`.
- 4 The sign immediately succeeds the value and `currency_symbol`.

Example:

```
#include<locale.h>
```

```
#include<stdio.h>
int main(void)
{
    struct lconv locale_structure;
    struct lconv *locale_ptr=&locale_structure;

    locale_ptr=lcoaleconv();
    printf("Decimal point:                %s",locale_ptr->decimal_point);
    printf("Thousands Separator:            %s",locale_ptr->thousands_sep);
    printf("Grouping:                            %s",locale_ptr->grouping);
    printf("International Currency Symbol:      %s",locale_ptr->int_curr_symbol);
    printf("Currency Symbol:                    %s",locale_ptr->currency_symbol);
    printf("Monetary Decimal Point:              %s",locale_ptr->mon_decimal_point);
    printf("Monetary Thousands Separator:        %s",locale_ptr->mon_thousands_sep);
    printf("Monetary Grouping:                  %s",locale_ptr->mon_grouping);
    printf("Monetary Positive Sign:              %s",locale_ptr->positive_sign);
    printf("Monetary Negative Sign:              %s",locale_ptr->negative_sign);
    printf("Monetary Intl Decimal Digits:        %c",locale_ptr->int_frac_digits);
    printf("Monetary Decimal Digits:            %c",locale_ptr->frac_digits);
    printf("Monetary + Precedes:                %c",locale_ptr->p_cs_precedes);
    printf("Monetary + Space:                   %c",locale_ptr->p_sep_by_space);
    printf("Monetary - Precedes:                %c",locale_ptr->n_cs_precedes);
    printf("Monetary - Space:                   %c",locale_ptr->n_sep_by_space);
    printf("Monetary + Sign Posn:                %c",locale_ptr->p_sign_posn);
    printf("Monetary - Sign Posn:                %c",locale_ptr->n_sign_posn);
}
```
